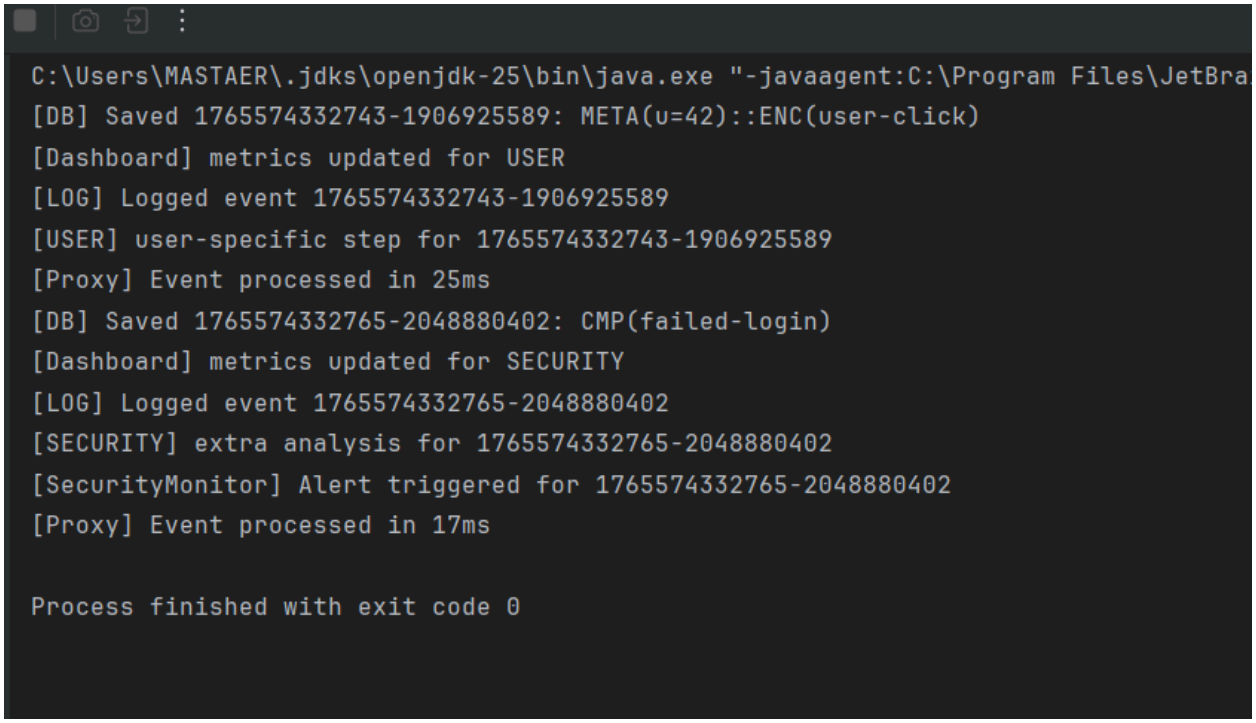
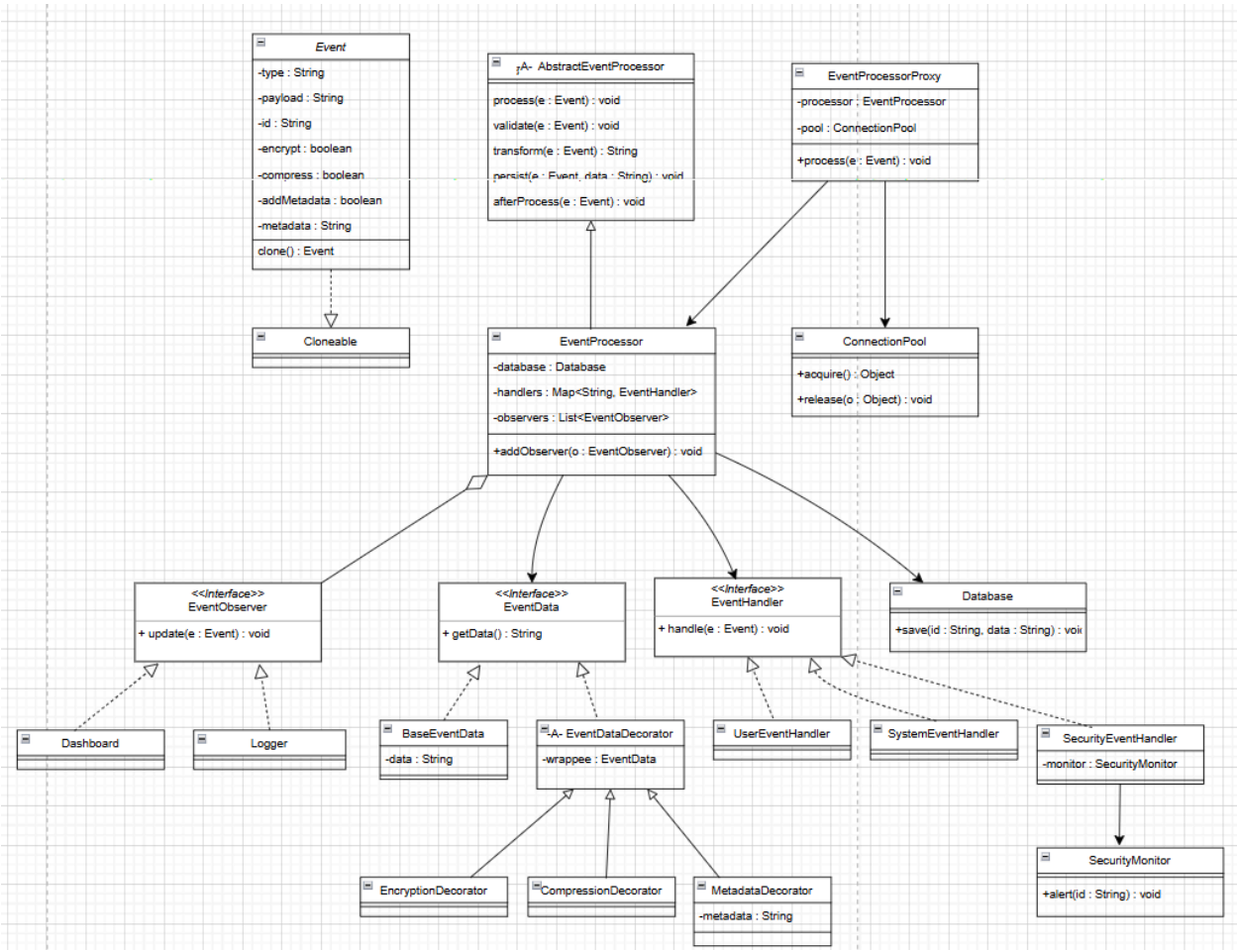
Assignment 2 Advace
Report:
Output:





Design Decision Document:

Template Method Pattern

1.Why each design pattern was chosen:

Template Method Pattern was chosen because the event handling process always goes through the same stages (verification, conversion, storage, and post-processing), with the details of each stage differing.

2.What alternatives you considered:

-The alternative was to put all the processing logic inside a single method, but this resulted in complex and difficult-to-maintain code.

-A chain of responsibility was considered, but it is more complex than what is needed.

3.How the patterns interact:

-The template method defines the processing sequence.

-Internal steps use Decorator for conversion and Observer for notifications.

4.How your architecture improves scalability, flexibility, and maintainability:

-Improved maintenance by separating the steps.

-Easier expansion without affecting the overall processing sequence.

Decorator Pattern:
1.Why each design pattern was chosen:
We chose Decorator Pattern to add encryption, compression, and metadata dynamically without using if/else.
2.What alternatives you considered:
Strategy Pattern with Pipeline, but less flexible because more than one transformation overlaps with another.
3.How the patterns interact:
-Decorators are used within the transform step of the Template Method.
-Each Decorator adds a behavior on top of the previous behavior.
4.How your architecture improves scalability, flexibility, and maintainability:
-Adding a new transformation is done simply by creating a new Decorator.
-The code becomes easier to maintain and less complex.

Observer Pattern:
1.Why each design pattern was chosen:
We chose Observer Pattern to separate the EventProcessor from the Dashboard and Logger.
2.What alternatives you considered:
Direct summoning is possible, but due to high coupling, expansion is difficult.
3.How the patterns interact:
-The EventProcessor notifies Observers when processing is complete.
-The Dashboard and Logger receive the update without knowing the processing details.
4.How your architecture improves scalability, flexibility, and maintainability:
-We can add a new Observer without modifying the EventProcessor.
-The interrelationship between classes has decreased.

Proxy Pattern:
1.Why each design pattern was chosen:
We chose Proxy Pattern to isolate shared tasks such as execution time measurement and resource management.
2.What alternatives you considered:
Merging these tasks into an EventProcessor violates the principle of single responsibility.
3.How the patterns interact:
-The proxy receives the event and then delegates it to the EventProcessor.
-It manages the connection pool before and after execution.
4.How your architecture improves scalability, flexibility, and maintainability:
-The core code is cleaner and easier to maintain.

-We can add new shared tasks within the Proxy without modifying the EventProcessor.

Connection Pool Pattern:
1.Why each design pattern was chosen:
-We chose the Connection Pool Pattern because creating and closing resources (such as connections) with each event is performance-intensive.
-This pattern allows for the reuse of existing resources instead of creating new ones each time.
-This contrasts with the more practical design we use in large, high-load systems.
2.What alternatives you considered:
-Creating a new connection for each event (inefficient).
-Using a single, fixed connection (bottleneck).
3.How the patterns interact:
-We use the Connection Pool within a proxy to manage resources.
-The resource is taken before execution and returned after completion.
4.How your architecture improves scalability, flexibility, and maintainability:
-Resource reuse improved performance.
-It increased scalability as the number of events increased.

Prototype Pattern:
1.Why each design pattern was chosen:
-We chose the Prototype Pattern because it's an event object that might need to be copied during processing or testing without affecting the original.
-This pattern allows creating new copies of the object at a lower cost than recreating it from scratch, especially when it has multiple properties.
-It helps maintain the original event state when applying different processes or tests to independent copies.
2.What alternatives you considered:
-Copy Constructor, but it's less flexible when changing class properties.
-Builder Pattern, but it's designed for creating, not copying, patterns.
3.How the patterns interact:
-We use Prototype when we need to process a copy of an event.
-It works independently of other patterns.
4.How your architecture improves scalability, flexibility, and maintainability:
-It increases flexibility by enabling work on multiple instances of the same event.
-It reduces collateral errors resulting from modifying the same object.
-It simplifies code maintenance when developing the Event class structure.

Issue Analysis Report:

All issues you identified in the original code:

-The EventProcessor contains a large method that performs various tasks such as validation, conversion, storage, and notifications.

-It uses if/else conditions to handle different event types.

-It mixes conversion logic (encoding, compression, metadata) with the underlying processing logic.

-It exhibits high coupling between the EventProcessor and both the Dashboard and Logger.

-It lacks a clear resource management mechanism.

-It does not support secure copying of event objects.

Why each issue is problematic:

-Multiple responsibilities within a single method make the code difficult to understand and maintain.

-Using if/else makes the code unscalable and violates the Open/Closed principle.

-Mixing conversion logic with processing increases complexity and makes adding new functionality difficult.

-High coupling reduces flexibility and means any change affects multiple parts of the system.

-Poor resource management causes performance issues when the number of events increases.

-The lack of copy support makes modifying events during processing susceptible to side effects.

How your refactoring resolves the issue:

-We used the Template Method Pattern to break down the processing into clear and specific steps.

-We used the Decorator Pattern to isolate the transformation logic and eliminate the need for if/else conditions.

-We used the Observer Pattern to separate the EventProcessor from the Dashboard and Logger.

-We used a Proxy Pattern to isolate shared tasks such as resource management and time measurement.

-We used a Connection Pool Pattern to improve resource management and increase performance.

-We used a Prototype Pattern to enable secure event copying without affecting the original.